configuration errors are more difficult to catch, but should be detected during a pre-installation check, ahead of the actual software being installed, so that the upgrade does not fail. Environmental dependencies are one of the most difficult upgrade errors to catch. Detecting dependencies relies upon the inputs from the software maintainers, and metadata on the system, such as the software catalogue inventory. Many operating systems provide packaging managers and software installation tools, which do a pre-installation check for the various dependencies, and allow the upgrade to go ahead only if all the pre-requisites are satisfied. Data-access errors, such as those arising from mismatches in database schema, can be caught by doing pre-installation testing with real-life workloads. A detailed discussion of the various software upgrade faults and their classification can be found in the paper "Why do upgrades fail, and what can we do about it?" available at *http://bit.ly/je5Oix*.

Note that there are different types of software upgrades that can be used on a large system that employs multiple host nodes interacting with each other. One is known as the rolling upgrade, where each node is upgraded independently, one at a time, and then added back to the system. Another process is the Big Flip upgrade, where half the nodes in the system are taken out and upgraded to the new software, while the other 50 per cent of the nodes in the system continue to service incoming requests. The upgraded nodes are added back to the system, while taking out the remaining 50 per cent of the nodes for upgrade. While a rolling upgrade results in a small perturbation to the system, it requires that the system allows for co-existence of old and new software. On the other hand, Big Flip can result in a large dip in service response times (since 50 per cent of the nodes are taken out for the upgrade), but does not require that the system should support the co-existence of old and new software. For instance, an incompatible database schema change cannot be applied as a rolling upgrade, whereas it can be applied as a Big Flip upgrade.

Offline v/s online software upgrades

An important point to consider when performing a software upgrade is whether the application or system needs to be brought down in order to apply the upgrade. Based on whether downtime is required, we can classify the upgrade as online (does not require the system/application to be brought down during the upgrade) or offline (requires downtime of the system/application being upgraded). Online upgrades allow interruption-free service even during the upgrade, though possibly with reduced performance/functionality. Since online upgrades support zero downtime for the end-user, they have been a major topic of research over the past few years. While there is considerable complexity in patching a user application dynamically without any interruption in service, consider the complexity involved in patching an entire operating system dynamically without requiring a reboot. However, recent advances in the research on dynamic software updates have resulted in the development of tools that allow just this. In next month's column, we will look at Ksplice, which supports dynamically patching Linux kernels. Interested readers can look up the Ksplice paper at *http://www.ksplice.com/doc/ksplice.pdf*.

This month's takeaway question

It is centred on dynamic software updates, and is a question to ponder over, rather than a programming problem. Can you think of the scenarios in which a software application cannot be patched dynamically? Can you write a tool which can verify whether a dynamically patched application is indeed performing correctly?

My 'must-read book/article' for this month

This month's recommendation comes from one of our readers, Ms Raji Sundararajan. Raji is a final-year Computer Science student. She recommends an article that recently appeared at the Scala Days 2011 conference, titled 'Loop Recognition in C++/Java/Go/Scala', available at *http://bit.ly/jUo2rr*. This interesting paper compares the performance of a graph algorithm typically used in compiler optimisations to benchmark the four languages. It shows that while C++ does beat the other implementations in terms of performance, it needed considerable tuning, and is dismal in terms of out-of-the-box performance. Thanks, Raji, for your suggestion.

If you have a favourite programming book/article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful, so I can mention it in the column. This would help many readers who want to improve their coding skills.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com*. Till we meet again next month, happy programming, and here's wishing you the very best! END

By: Sandya Mannarswamy

The author is an expert in systems software and works at Hewlett-Packard India. Her interests include compilers, virtualisation technologies and software development tools. If you are preparing for systems software interviews, you may find it useful to visit Sandya's website http://www.tech-pathshala.com, which offers practice interviews and training for systems software jobs, and *tech-pathshala*, her system software training group at LinkedIn (*http://www.linkedin.com/groups?home=&gid=3813966&trk=anet_ug_hm*).